

Data Analysis with Python 2024–2025

Parte 3: NumPy (Parte 2)

Caderno de Exercícios

Tradução e Adaptação: University of Helsinki — MOOC.fi

Nota Importante

Este material é uma tradução e adaptação baseada no curso *Data Analysis with Python 2024-2025*, oferecido pela **The University of Helsinki MOOC Center**.

Conteúdo

1 Exercícios - Capítulo 3 (Parte 2)

Exercício 3.1 – Comparação de colunas

Sumário

- **Objetivo:** Escreva a função `column_comparison`, que recebe um array bidimensional. A função deve retornar um novo array contendo as linhas em que o valor da segunda coluna é maior do que o valor da penúltima coluna. Pode-se assumir que a entrada possui pelo menos duas colunas. Não use loops; use operações vetorizadas.

Exercício 3.2 – Primeira metade e segunda metade

Sumário

- **Objetivo:** Escreva `first_half_second_half`, que recebe um array de forma $(n, 2m)$. Para cada linha, calcule a soma dos primeiros m elementos e a soma dos últimos m elementos. Retorne as linhas nas quais a primeira soma é maior. Sua solução deve chamar `np.sum` (ou o método equivalente) exatamente duas vezes.

Exercício 3.3 – Mais frequente primeiro

Sumário

- **Objetivo:** Escreva a função `most_frequent_first`, que recebe um array bidimensional e o índice c de uma coluna. As linhas devem ser reorganizadas de forma que primeiro apareçam as linhas cujo elemento na coluna c seja o mais frequente, depois as do segundo elemento mais frequente, e assim por diante. Os valores das outras colunas não devem influenciar a ordenação.
- **Dica:** A ordem entre linhas que pertencem ao mesmo grupo de frequência pode ser arbitrária. A função `np.unique` pode ser útil.

Exercício 3.4 – Potência de matriz

Sumário

- **Objetivo:** Implemente a funcionalidade de `numpy.linalg.matrix_power`. Dada uma matriz quadrada a e um inteiro não negativo n , calcule $a^n = a @ a @ \dots @ a$ (n vezes). Quando $n = 0$, o resultado deve ser `np.eye(m)`.
- **Implementação:** Use a função `reduce` e uma expressão geradora. Depois, estenda a função para potências negativas: nesse caso, a^n é definido como a potência correspondente da matriz inversa de a . A entrada pode ser considerada invertível.

Exercício 3.5 – Correlação

Sumário

- **Contexto:** Considere o conjunto de dados Iris, contendo: 1. comprimento da sépala, 2. largura da sépala, 3. comprimento da pétala e 4. largura da pétala.
- **Parte 1:** Calcule a correlação de Pearson entre comprimento da sépala e comprimento da pétala usando `scipy.stats.pearsonr`. A função `lengths` deve carregar os dados e retornar a correlação.
- **Parte 2:** Calcule todas as correlações em um array (4, 4) usando `np.corrcoef`. Determine qual par de variáveis possui a maior correlação.

Exercício 3.6 – Encontro de retas

Sumário

- **Objetivo:** Escreva a função `meeting_lines`, que recebe os coeficientes de duas retas e retorna o ponto onde elas se encontram. As equações são: $y = a_1x + b_1$ e $y = a_2x + b_2$.
- **Dica:** Use `np.linalg.solve` e crie uma função principal para testar a solução.

Exercício 3.7 – Encontro de planos

Sumário

- **Objetivo:** Escreva a função `meeting_planes`, que recebe os coeficientes de três planos e retorna o ponto onde os três se encontram. As equações são: $z = a_1y + b_1x + c_1$, $z = a_2y + b_2x + c_2$ e $z = a_3y + b_3x + c_3$.
- **Dica:** Use `np.linalg.solve`.

Exercício 3.8 – Retas quase concorrentes

Sumário

- **Objetivo:** No exercício anterior de encontro de retas, existe um problema quando as retas não se encontram. Estenda a solução para indicar o ponto de encontro quando ele existir e, caso contrário, encontrar o ponto mais próximo de ambas usando `numpy.linalg.lstsq`.

Créditos: Este conteúdo foi originalmente criado pela *The University of Helsinki MOOC Center* para o curso *Data Analysis with Python*.